

Calibrated Loop Control for Multi-Hop QA with Small Language Models

Pin-Hung Lin

National Cheng Kung University

f84076029@gs.ncku.edu.tw

<https://orcid.org/0009-0000-1916-7829>

Te-Lun Yang

National Taiwan University

d12944007@ntu.edu.tw

<https://orcid.org/0000-0002-3351-1785>

Abstract

Small language models (SLMs) often stop multi-hop question-answering loops too early, retrieve unnecessarily, or misuse tools. Most adaptive retrieval methods rely on the generator’s signals. When should an external calibrated controller take over? We study Laya, a compact non-autoregressive decision model with calibrated outputs. At each hop, it predicts whether the evidence is sufficient, what to do next, and how much evidence is missing. Our 2×2 factorial design uses a fixed decomposition loop with two evidence sources and two controllers. Evidence comes from closed-book recall or retrieval-augmented generation (RAG) with re-ranking. The generating large language model (LLM) or Laya controls the loop. We evaluate five Qwen3.5 models (0.8B to 27B parameters) on HotpotQA, 2WikiMultihopQA, and MuSiQue, using 200 questions per configuration. Trained on weak labels, Laya achieves 0.9271 held-out accuracy on evidence sufficiency. Retrieval improves exact match (EM) by up to 43.5 percentage points. Under RAG, Laya raises EM point estimates by 1.5–9.0 points for the 0.8B and 2B models but lowers EM by 3.5–15.5 points for the 4B–27B models. It reduces prompt tokens and hops throughout. Its most frequent error is stopping before all gold evidence is retrieved. The accuracy-efficiency trade-off of external control therefore depends on model size.

1. Introduction

Large language models (LLMs) can hallucinate, rely on outdated knowledge, and lack specialised facts. Retrieval-augmented generation (RAG) [9] provides the generator with passages from an external corpus. Combining dense search with a cross-encoder re-ranker further improves question-answering (QA) accuracy across many LLMs [16].

Multi-hop questions combine facts from multiple documents, for example to identify a bridge entity and ask about its properties [6, 13, 17]. A single retrieval often misses second-hop evidence. Agents therefore decompose

questions into sub-questions and gather evidence iteratively. Small language models (SLMs) running locally often struggle to control this loop. They may answer too early, retrieve after collecting enough evidence, or ask irrelevant sub-questions.

These control decisions are small, structured prediction problems: *Is the evidence sufficient? What should happen next? How much evidence is still missing?* Using the generator ties decision quality to model size and consumes tokens at every step. Laya [3] is a non-autoregressive "System 1" decision model. It answers multiple-choice, yes/no, and ordinal questions in one forward pass, returning a probability for each option. Training uses strictly proper scoring rules, and temperature scaling calibrates these probabilities. Most adaptive retrieval methods use the generator’s signals or decide once per query (Section 2). When external calibrated control should replace the generator’s control, and how this depends on generator size, remain open questions.

Our 2×2 factorial design uses a shared decomposition loop to isolate the controller’s effect. We vary the *evidence source* (closed-book recall or RAG with dense retrieval and re-ranking) and the *loop controller* (the LLM or Laya). Laya’s decisions are enforced at every hop, rather than offered as an optional tool. Thinking mode is disabled for all models. We evaluate five Qwen3.5 sizes [11] on three multi-hop benchmarks, with 200 questions per configuration ($5 \times 4 \times 3 = 60$ configurations). We report exact match (EM), token-level F1, cost, paired bootstrap 95% confidence intervals (CIs), and the controller’s Brier score and expected calibration error (ECE). We ask three research questions (RQs).

- **RQ1 (Accuracy).** With the decomposition loop fixed, does replacing the generator’s control with a calibrated external decision model change multi-hop QA accuracy (EM and F1)? Does the effect depend on the evidence source?
- **RQ2 (Cost).** How does external control affect tokens, hops, end-to-end latency, and tool-call reliability per question?
- **RQ3 (Scale).** How do these effects change with generator scale, from 0.8B to 27B parameters?

Our four contributions address these research questions as follows.

- A factorial framework separating evidence and controller effects within one loop, with wrong answers classified at the tool, retrieval, stopping, or answering stage (RQ1).
- A weak-supervision method that simulates agent states from supporting-paragraph annotations to train a calibrated multi-hop controller for RQ1–RQ3.
- An analysis of tokens, hops, latency, and tool-call reliability under explicit control in local SLM agents (RQ2).
- An evaluation of five Qwen3.5 sizes on three datasets showing that the accuracy effect of external control reverses with generator scale (RQ3).

Sections 2–4 cover related work, methods, and results. Section 5 answers the RQs and discusses limitations. Section 6 concludes the paper.

2. Related Work

Retrieval Pipelines. RAG [9] supplies external passages to a generator. Yang *et al.* [16] combined BGE-M3 embeddings [2], an exact inner-product index [5], and a BGE cross-encoder re-ranker [1, 2]. They reported accuracy gains for nearly all of 17 LLMs on two Chinese QA benchmarks. We adopt this pipeline and study who decides when to retrieve and stop.

Multi-Hop QA and Agents. HotpotQA [17], 2Wiki-MultihopQA [6], and MuSiQue [13] require facts across Wikipedia paragraphs and label the supporting ones. The first two use EM and token-level F1, while MuSiQue uses answer F1. We report EM and F1 on all three and use their annotations to build corpora, derive weak labels, and classify errors. AutoGen [15] and its continuation AG2 [14] let an LLM call tools across turns. Loops usually end through the generator’s answer tool or a step limit. Tool-call reliability varies with model size.

Adaptive and Dynamic Retrieval. FLARE [8] retrieves when an upcoming sentence has low-confidence tokens. Adaptive-RAG [7] uses query complexity to choose no, single-step, or iterative retrieval. DRAGIN [12] decides when and what to retrieve from the generator’s information needs. SeaKR [18] uses uncertainty in the generator’s internal states to trigger retrieval. These methods use the generator’s signals or decide once per query. S2G-RAG [10] checks evidence sufficiency at each turn. RAG-on-a-Diet [4] trains a reinforcement-learning agent to select the smallest model that can handle each hop and to stop at a confidence threshold.

Calibrated Decision Models. Given a textual state and a question, Laya [3] returns option probabilities in one forward pass. It supports *choice*, *noul* (yes/no, returning $P(\text{true})$), and *score* (ordinal) questions. Miscalibrated sufficiency probabilities can trigger early or late stops. We therefore report the Brier score and ECE.

Table 1. The four conditions of the 2×2 factorial design. All conditions share the loop of Algorithm 1.

Condition	Evidence source e	Controller c
Recall+LLM	RECALL (closed-book)	LLM
RAG+LLM	RAG (retrieve + re-rank)	LLM
Recall+Laya	RECALL (closed-book)	LAYA
RAG+Laya	RAG (retrieve + re-rank)	LAYA

Positioning. Deciding when to retrieve and when to stop is an established problem, and we do not claim a new decision layer for it. Our controller is calibrated, non-autoregressive, separate from the generator, and trained only on weak labels. Our factorial design changes only the evidence source and controller. RAG-on-a-Diet selects among generators to reduce cost. We instead vary generator size to study how it changes the accuracy-efficiency trade-off of external control.

3. Method

3.1. Problem Setting and 2×2 Factorial Design

For a multi-hop question q from dataset d , the agent uses an LLM M and corpus $\mathcal{C}_d$. Its state $s = (q, Q_{\text{sub}}, E)$ tracks the question, sub-questions, and collected evidence. The *evidence source* $e \in \{\text{RECALL}, \text{RAG}\}$ determines where evidence comes from. The *loop controller* $c \in \{\text{LLM}, \text{LAYA}\}$ decides when to stop and answer. Their combinations give the four conditions in Table 1.

The design separates the two main effects and their interaction. Recall+LLM also uses the loop: the LLM asks sub-questions, answers from memory, and decides when to stop. It does not give a single-shot closed-book answer. Differences between LLM and LAYA therefore reflect who stops the loop and which tools are available, rather than iterative decomposition.

3.2. Shared Decomposition Loop

Algorithm 1 shows the shared loop. At each hop, the controller can stop. Otherwise, the LLM asks a sub-question and collects evidence through retrieval or recall. The LLM answers from this evidence when the controller stops or the loop reaches $H_{\text{max}} = 4$ hops.

Controller. When $c = \text{LAYA}$, Laya runs at each hop’s start. If its sufficiency probability is $p_{\text{suf}} \geq 0.5$, the loop ends and the LLM must answer. Otherwise, its predicted next action and remaining hops guide the LLM, with only `ask_sub_question` available. When $c = \text{LLM}$, the LLM can call `ask_sub_question` or stop through `final_answer`. We count the stopping hop, so hops equal decisions. Each question has a 300 s budget.

Evidence and failures. When $e = \text{RAG}$, the dense retriever uses the sub-question to find the top-20 passages in

Algorithm 1 Decomposition loop shared by the four conditions. Retrieved passages are de-duplicated by passage id and capped at $|E| \leq 8$.

Require: question q ; evidence source $e \in \{\text{RECALL}, \text{RAG}\}$; controller $c \in \{\text{LLM}, \text{LAYA}\}$;
 $H_{\max} = 4$

- 1: $Q_{\text{sub}} \leftarrow []$; $E \leftarrow []$
- 2: **for** $h = 1, \dots, H_{\max}$ **do**
- 3: $d_h \leftarrow \emptyset$ $\triangleright$ no hint unless $c = \text{LAYA}$
- 4: **if** $c = \text{LAYA}$ **then**
- 5: $d_h \leftarrow \text{Laya}(\text{BuildState}(q, Q_{\text{sub}}, E))$
- 6: **if** $d_h.p_{\text{suf}} \geq 0.5$ **then break** $\triangleright$ enforced
- 7: $q_h \leftarrow M.\text{SubQuestion}(q, Q_{\text{sub}}, E, d_h)$
- 8: **if** $c = \text{LLM}$ **and** M called `final_answer` **then**
- 9: **break**
- 10: append q_h to Q_{sub}
- 11: **if** $e = \text{RAG}$ **then**
- 12: $E \leftarrow E \cup \text{Rerank}(q_h, \text{Dense}(q_h, 20))[1:3]$
- 13: **else**
- 14: $E \leftarrow E \cup \{M.\text{Recall}(q_h)\}$
- 15: **return** $M.\text{Answer}(q, E)$

$\mathcal{C}_d$. The re-ranker’s top-3 passages are added to E . When $e = \text{RECALL}$, we add the LLM’s closed-book answer to the sub-question. A reply without a valid call to an available tool, including a plain-text reply, ends the question as `invalid_tool`. Exceeding the time budget gives `timeout`. Other failures receive `error` or `oom`. All count as incorrect.

3.3. Laya as the Decision Layer

At each hop, Laya answers three fixed English questions about the state: whether to stop, what to do next, and how much evidence is missing. Each uses a different question type.

- `sufficient (noul)`: is the evidence sufficient for the original question? Laya returns $p_{\text{suf}} = P(\text{true})$.
- `next_action (choice)`: (A) answer now; (B) form one more sub-question and search; (C) rephrase the query and search again.
- `remaining_hops (score)`: how many key evidence passages are missing? The options are 0, 1, or 2 (two or more).

State construction. `BUILDSTATE` converts the question, sub-questions, and evidence into text within a 900-token limit, keeping only the first two sentences of each item. It uses the same format for training and inference.

3.4. Weak Supervision by State Simulation

The datasets lack human labels for these three decisions, but their training splits identify the supporting paragraphs.

For a training question with gold paragraphs G and distractors D , we simulate four kinds of states: (1) *question only*, $E = \emptyset$; (2) *partial gold*, a non-empty proper subset of G plus 0–2 passages from D ; (3) *all gold*, all of G plus 0–2 passages from D , shuffled; and (4) *distractors only*.

Let $m = |G \setminus E|$ be the number of gold paragraphs missing from the evidence. The labels are

$$\text{sufficient} = \mathbf{1}[m = 0], \quad (1)$$

$$\text{next_action} = \begin{cases} \text{A} & \text{if } m = 0, \\ \text{C} & \text{if } E \neq \emptyset \text{ and } E \cap G = \emptyset, \\ \text{B} & \text{otherwise,} \end{cases} \quad (2)$$

$$\text{remaining_hops} = \min(m, 2). \quad (3)$$

Missing gold evidence calls for another decomposition step. If the evidence contains only distractors, the query should be rephrased instead.

With a fixed seed, we sample 800, 100, and 100 training questions per dataset for decision-layer training, calibration, and held-out evaluation. We target about 30,000 training decisions. Before the main runs, `sufficient` and `next_action` must each reach 0.75 accuracy and exceed the zero-shot base checkpoint (evaluated only here) by at least 0.20. Mean absolute error (MAE) on `remaining_hops` must be at most 0.5.

3.5. Evaluation Protocol

Our primary measure is EM after official answer normalisation. We also report token-level F1 (for MuSiQue, the maximum over the answer and its aliases). For each question, we record end-to-end latency, device-wide peak GPU memory, LLM prompt and output tokens, hops, and Laya-call latency. For the decision layer, we report accuracy, Brier score, ECE, and latency per decision. EM differences include paired percentile-bootstrap 95% CIs based on 10,000 resamples of the 200 questions.

3.6. Implementation Details

Models and Datasets: We use five instruction-tuned Qwen3.5 models [11] (0.8B, 2B, 4B, 9B, and 27B parameters) with thinking disabled, an 8192-token context, greedy decoding, and seed 42. All run on one 24 GB consumer GPU (software, checkpoints, and licenses are in Appendix A). We evaluate HotpotQA [17] in the distractor setting, 2WikiMultihopQA [6] (2Wiki), and the answerable subset of MuSiQue [13]. For each dataset, we sample 200 test questions from the validation split and the 800/100/100 decision-data questions from the training split. No test questions enter decision-layer training, calibration, or held-out evaluation. Before the main runs, each model answers 20 fixed tool-calling probe items. We retain all models.

Retrieval: Following [16], we embed passages and sub-questions with BGE-M3 [2], retrieve passages by exact inner-product search, and re-score them with the BGE cross-encoder re-ranker [1]. Each dataset’s corpus combines all supporting and distractor paragraphs from its 200 test questions. We remove duplicates and use one chunk per paragraph. The embedder, re-ranker, and Laya share the GPU with the generator.

Controller: We fine-tune one shared decision layer for all datasets from the multilingual Laya checkpoint [3]. Training uses Laya’s reinforcement learning with strictly proper scoring rules [3]. We then fit one temperature per question type on the calibration split (details are in Appendix A).

4. Experimental Results

4.1. Setup

Each of the 60 configurations ran once on 200 questions, giving 12,000 evaluations (Section 3.6 and Table 1). EM and F1 are percentages, and differences are percentage points (pp). One correct answer adds 0.5 pp EM. Sections 4.3 and 4.5 address RQ1 and RQ2, and paragraphs marked RQ3 examine model size. Section 4.4 analyses errors. Section 5 answers the RQs.

4.2. Function-Calling Probe and Decision-Layer Quality

Every model made valid tool calls on all 20 probe items (Table 2, Panel A), with mean latency from 134 ms (0.8B) to 836 ms (27B). This probe used one tool and single-hop questions. It did not predict the main results, where the 0.8B model’s `invalid_tool` rate under LLM control was 0.290–0.550. Other models stayed at or below 0.030 in every configuration.

The decision layer met all thresholds in Section 3.4 (Table 2, Panel B). Accuracy reached 0.9271 on `sufficient` (+0.3356 over zero-shot) and 0.8333 on `next_action` (+0.4859). MAE on `remaining_hops` was 0.2013. For `sufficient`, the Brier score fell from 0.5551 to 0.1343 and ECE from 0.2271 to 0.0626. All fitted temperatures reached 5.0, the upper bound of the search range. A call answering all three questions had a median latency of 12.6 ms (zero-shot: 13.2 ms).

4.3. Main Results (RQ1)

Table 3 (Panel A) reports EM and F1, and Figs. 1–3 show EM by model size. The evidence source has the largest effect. RAG+LLM exceeds Recall+LLM for every model and dataset, by 19.0–43.5 pp EM on HotpotQA, 3.5–36.5 pp on 2Wiki, and 3.5–30.0 pp on MuSiQue. RAG+LLM with 27B performs best throughout (EM 67.5, 68.5, and 37.0, respectively). No model exceeds 10.0 EM without retrieval on

Table 2. Function-calling probe with mean latency per call (Panel A). Decision-layer quality on 4,032 held-out decisions (Panel B), fine-tuned (FT) versus zero-shot (ZS): Brier for `sufficient` and `next_action`, MAE for `remaining_hops`.

Panel A: Function-calling probe					
Model	27B	9B	4B	2B	0.8B
Items	20	20	20	20	20
Valid rate	1.00	1.00	1.00	1.00	1.00
Latency (ms)	836	309	235	178	134
Panel B: Decision layer on held-out decisions					
Question		Acc.	Brier	ECE	MAE
<code>sufficient</code>	FT	0.9271	0.1343	0.0626	—
	ZS	0.5915	0.5551	0.2271	—
<code>next_action</code>	FT	0.8333	0.3024	0.1339	—
	ZS	0.3475	0.7587	0.2119	—
<code>remaining_hops</code>	FT	0.8177	—	0.1414	0.2013
	ZS	0.3036	—	0.3750	0.8783
Overall	FT	0.8594	—	0.1114	—
	ZS	0.4142	—	0.2703	—

Table 3. Main results, 200 questions per cell. Panel A: EM / F1 (%). Panel B: controller effects on EM (pp) with paired bootstrap 95% CIs: $\Delta_{\text{RAG}} = \text{RAG+Laya} - \text{RAG+LLM}$, and the interaction $\Delta_{\text{int}} = \Delta_{\text{RAG}} - \Delta_{\text{Rec}}$, where $\Delta_{\text{Rec}} = \text{Recall+Laya} - \text{Recall+LLM}$ (Appendix B).

Panel A: EM / F1 (%)							
Model	Condition	HotpotQA		2Wiki		MuSiQue	
		EM	F1	EM	F1	EM	F1
27B	Recall+LLM	30.0	39.9	32.0	38.5	10.0	17.0
	RAG+LLM	67.5	79.6	68.5	76.0	37.0	50.3
	Recall+Laya	28.0	38.9	33.5	39.4	10.0	18.2
	RAG+Laya	61.0	74.0	56.0	60.9	31.5	41.3
9B	Recall+LLM	21.0	30.4	26.0	30.0	4.0	10.5
	RAG+LLM	64.0	75.3	61.0	69.0	32.0	43.6
	Recall+Laya	21.0	30.0	26.0	29.5	4.0	11.4
	RAG+Laya	60.5	72.3	46.5	52.2	25.5	36.2
4B	Recall+LLM	21.0	29.7	24.5	28.5	3.5	8.8
	RAG+LLM	64.5	76.9	60.0	68.8	33.5	44.9
	Recall+Laya	21.5	31.1	27.0	31.2	2.5	8.5
	RAG+Laya	56.5	70.1	44.5	50.3	22.0	33.0
2B	Recall+LLM	16.5	23.3	24.5	28.3	1.0	7.0
	RAG+LLM	40.0	53.3	35.0	39.6	9.5	15.3
	Recall+Laya	16.5	22.1	22.5	27.0	1.0	6.2
	RAG+Laya	47.5	60.4	36.5	41.8	13.5	20.1
0.8B	Recall+LLM	4.0	7.0	7.0	8.6	0.0	1.8
	RAG+LLM	23.0	29.1	10.5	13.1	3.5	6.7
	Recall+Laya	9.5	13.9	15.0	17.5	0.5	4.2
	RAG+Laya	30.5	40.3	19.5	23.2	10.0	13.6
Panel B: EM effect (pp), estimate [paired bootstrap 95% CI]							
Model	Dataset	Δ_{RAG}		Δ_{int}			
27B	HotpotQA	−6.5	[−10.5, −3.0]	−4.5	[−9.5, 0.5]		
	2Wiki	−12.5	[−18.0, −7.0]	−14.0	[−20.5, −7.5]		
	MuSiQue	−5.5	[−11.0, 0.0]	−5.5	[−11.5, 0.5]		
9B	HotpotQA	−3.5	[−7.5, 0.5]	−3.5	[−9.0, 2.0]		
	2Wiki	−14.5	[−20.5, −9.0]	−14.5	[−21.0, −7.5]		
	MuSiQue	−6.5	[−11.5, −1.5]	−6.5	[−12.0, −1.5]		
4B	HotpotQA	−8.0	[−12.5, −3.5]	−8.5	[−14.5, −2.5]		
	2Wiki	−15.5	[−21.5, −9.5]	−18.0	[−26.0, −9.5]		
	MuSiQue	−11.5	[−16.5, −6.5]	−10.5	[−15.5, −5.5]		
2B	HotpotQA	7.5	[3.0, 12.5]	7.5	[2.0, 13.0]		
	2Wiki	1.5	[−3.5, 6.5]	3.5	[−3.0, 10.0]		
	MuSiQue	4.0	[0.0, 8.0]	4.0	[0.0, 8.5]		
0.8B	HotpotQA	7.5	[2.0, 13.0]	2.0	[−5.0, 9.0]		
	2Wiki	9.0	[4.5, 14.0]	1.0	[−5.5, 7.5]		
	MuSiQue	6.5	[3.5, 10.0]	6.0	[2.5, 9.5]		

MuSiQue, the hardest dataset.

Model size (RQ3). Under recall, Laya changes EM by −2.0 to +2.5 pp for the 2B–27B models, equivalent to

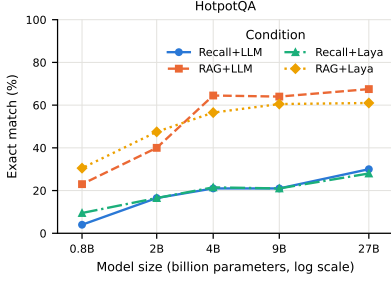


Figure 1. EM by model size, HotpotQA.

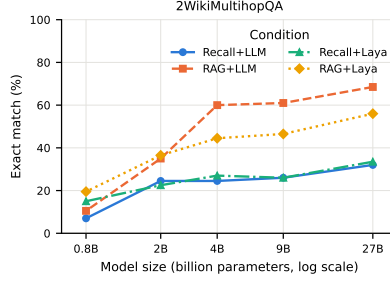


Figure 2. EM by model size, 2Wiki.

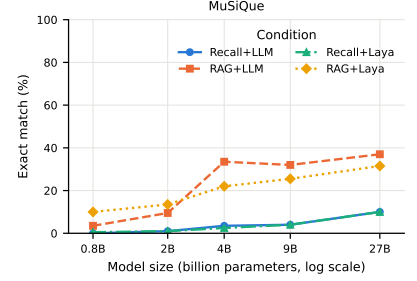


Figure 3. EM by model size, MuSiQue.

at most five questions. The 0.8B model’s EM point estimates rise by 5.5, 8.0, and 0.5 pp on HotpotQA, 2Wiki, and MuSiQue. Under retrieval, the effect depends on size. RAG+Laya falls below RAG+LLM by 3.5–15.5 pp EM on every dataset for the 4B, 9B, and 27B models. The largest drops occur on 2Wiki (−15.5, −14.5, and −12.5 pp, respectively). For the 2B and 0.8B models, RAG+Laya gives the highest EM point estimates on every dataset, exceeding RAG+LLM by 1.5–9.0 pp. On HotpotQA, EM rises from 40.0 to 47.5 for 2B and from 23.0 to 30.5 for 0.8B. The interaction point estimate is negative for all nine combinations of larger models and datasets, and positive for all six with smaller models. F1 changes in the same direction.

Sampling uncertainty. Under RAG, the CI for Laya’s effect excludes zero in eleven of the fifteen combinations of models and datasets (Table 3, Panel B). These include 0.8B on all three datasets (lower bounds of 2.0–4.5 pp), seven of the nine combinations with larger models, and 2B only on HotpotQA ([3.0, 12.5]). Interaction CIs exclude zero in six of nine combinations with larger models, all negative, and two of six with smaller models, both positive (2B on HotpotQA and 0.8B on MuSiQue). Appendix B lists intervals including zero and the controller effect under recall and the retrieval effect.

4.4. Error Analysis

Table 4 classifies wrong answers under retrieval by stage using gold supporting paragraphs (rules in Appendix B). Categories cover failed calls, complete evidence, premature stops, and retrieval misses after four hops. Under RAG+Laya, premature stops are the most frequent category for every model: 194–230 questions, or 45.4–65.0% of wrong answers, compared with 0–56 under RAG+LLM. Under RAG+LLM, the 4B–27B models most often fail despite having retrieved all gold evidence (47.9–58.5%). The 2B model mainly fails to retrieve the evidence within $H_{\max}$ (315 questions, 73.1%). The 0.8B model mainly fails through retrieval misses (247) and invalid tool calls (211). Laya reduces retrieval misses from 315 to 78 for 2B and from 247 to 108 for 0.8B. However, premature stopping becomes the main source of error for every model.

Table 4. Wrong answers under retrieval by stage, pooled over the three datasets (600 questions per row): invalid tool call (Inv.), time-out (T/O), error (Err.), retrieval miss after all hops (Miss), all gold evidence retrieved (Cov.), premature stop with gold evidence missing (Stop), and other (Oth.).

Model	Condition	Inv.	T/O	Err.	Miss	Cov.	Stop	Oth.
27B	RAG+LLM	1	0	0	72	147	34	0
	RAG+Laya	0	0	0	9	100	194	0
9B	RAG+LLM	3	0	1	89	137	56	0
	RAG+Laya	0	0	0	17	107	211	0
4B	RAG+LLM	0	0	0	105	166	13	0
	RAG+Laya	0	0	0	16	108	230	0
2B	RAG+LLM	1	0	0	315	115	0	0
	RAG+Laya	0	0	0	78	112	215	0
0.8B	RAG+LLM	211	0	0	247	67	1	0
	RAG+Laya	0	0	0	108	154	218	0

Appendix B gives the counts per dataset and a representative premature stop.

4.5. Efficiency (RQ2)

RAG+Laya uses fewer prompt tokens, output tokens, and hops than RAG+LLM throughout (Table 5). Mean prompt tokens fall from 2,776–6,067 to 1,565–2,329, and mean hops from 2.67–4.00 to 2.36–2.94. Under its own control with retrieval, 2B almost always reaches $H_{\max}$ (mean hops of 3.98, 4.00, and 4.00). For 0.8B–9B, RAG+Laya lowers p50 latency on every dataset (Fig. 4), for example from 1880 to 657 ms for 2B on HotpotQA. Laya calls for these models average 16.0–20.8 ms across conditions. All one-hop outcomes under LLM control are `invalid_tool` failures, not deliberate stops (Fig. 5). RAG+Laya most often stops at hop 2. Appendix B reports recall costs and p95 latency.

Model size (RQ3). The 27B model is the exception. Its Laya calls took 361.3–1261.4 ms on average. RAG+Laya had higher p50 latency than RAG+LLM on HotpotQA (13482 versus 11918 ms) and 2Wiki (11490 versus 11044 ms). With the 27B model, retrieval components, and Laya sharing the 24 GB GPU, peak memory use reached 23,526–24,461 MB. These slow calls are consistent with memory pressure rather than the decision model’s usual latency (a held-out median of 12.6 ms). Memory pressure

Table 5. Efficiency per model and condition, one panel per dataset: end-to-end p50 latency (ms), device-wide peak GPU memory (VRAM, MB), mean prompt (In) and output (Out) tokens per question, mean hops, and mean latency per Laya call (ms).

Model	Condition	Panel A: HotpotQA						Panel B: 2Wiki						Panel C: MuSiQue					
		p50	VRAM	In	Out	Hops	Laya	p50	VRAM	In	Out	Hops	Laya	p50	VRAM	In	Out	Hops	Laya
27B	Recall+LLM	7965	24461	1995	499	2.63	—	7882	24215	2101	290	3.00	—	9771	24390	2500	595	3.19	—
	RAG+LLM	11918	24429	3000	151	2.75	—	11044	24357	5490	136	3.19	—	16217	24106	4736	271	3.55	—
	Recall+Laya	17279	24366	1586	482	2.98	1261.4	13780	23526	1544	317	3.15	706.3	18126	24330	1667	439	3.18	840.2
	RAG+Laya	13482	24428	1606	68	2.57	595.3	11490	23647	2293	64	2.54	377.6	13170	24328	2187	76	2.81	361.3
9B	Recall+LLM	2605	21657	1856	309	2.56	—	2745	22092	1907	228	2.82	—	3000	21884	2096	353	2.95	—
	RAG+LLM	2022	21655	2939	104	2.67	—	3256	22409	5385	121	3.21	—	3028	21884	4628	177	3.52	—
	Recall+Laya	2676	21832	1509	344	3.02	19.1	2501	21919	1440	212	3.07	20.1	2770	21885	1579	243	3.20	19.7
	RAG+Laya	1570	22285	1675	71	2.62	16.3	1763	21871	2329	64	2.55	16.3	1731	22581	2324	80	2.89	17.0
4B	Recall+LLM	1843	19063	1934	476	2.55	—	1642	19117	1700	254	2.42	—	2960	19110	2461	414	3.08	—
	RAG+LLM	1961	19113	3445	122	3.06	—	2851	19116	6001	130	3.42	—	2791	19110	5018	146	3.75	—
	Recall+Laya	2221	19170	1569	509	2.92	18.6	2096	19111	1491	306	2.99	18.2	2445	19110	1734	321	3.15	18.4
	RAG+Laya	1095	19116	1565	68	2.52	16.1	1452	19109	2280	73	2.53	16.0	1408	19108	2255	80	2.88	16.5
2B	Recall+LLM	2733	15975	3196	526	3.79	—	2724	17123	3183	501	3.88	—	2826	17628	3363	491	3.98	—
	RAG+LLM	1880	15977	4202	166	3.98	—	2723	17124	6067	157	4.00	—	1818	17776	4275	156	4.00	—
	Recall+Laya	1277	15979	1287	315	2.67	16.8	1335	17223	1325	234	2.75	18.0	1494	17976	1699	290	3.10	18.3
	RAG+Laya	657	16810	1578	73	2.51	17.0	859	17324	1885	59	2.36	16.6	963	18077	2325	93	2.94	16.4
0.8B	Recall+LLM	1565	10227	2191	279	3.04	—	1443	14272	2044	262	2.90	—	1464	14250	1978	263	2.75	—
	RAG+LLM	1669	14306	2961	143	3.17	—	1947	14271	3459	125	2.83	—	1476	14251	2776	139	2.89	—
	Recall+Laya	828	14323	1345	192	2.75	20.8	712	14256	1124	167	2.62	17.4	741	14249	1328	193	2.73	17.2
	RAG+Laya	553	14296	1875	81	2.65	17.2	753	14250	1903	61	2.37	16.8	532	14249	1954	86	2.69	17.5

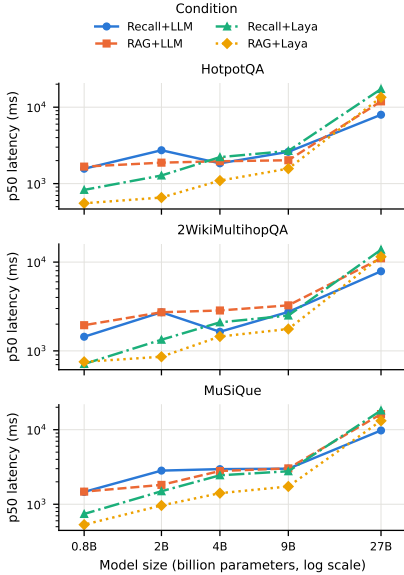


Figure 4. End-to-end p50 latency (log scale) by model size.

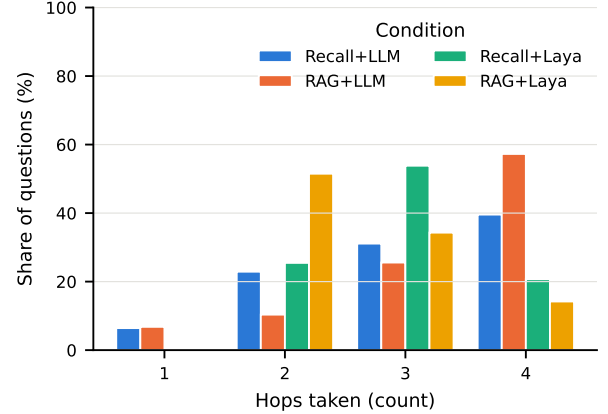


Figure 5. Hops per question by condition, pooled over models and datasets; the stopping hop is counted.

therefore complicates latency comparisons involving Laya at 27B.

5. Discussion

5.1. Answers to the Research Questions

RQ1 (Accuracy). Laya changed EM mainly under retrieval, with effects depending on model size. Under recall, EM changed by -2.0 to $+2.5$ pp for 2B–27B, suggesting that available evidence limits performance more than loop control. The 0.8B point estimates rose by up to 8.0 pp. Under retrieval, RAG+Laya exceeded RAG+LLM by 1.5–

9.0 pp in EM point estimates for 2B and 0.8B, but fell below it by 3.5–15.5 pp for 4B–27B (Table 3). CIs excluded zero for 0.8B on every dataset, seven of nine combinations with larger models, and 2B only on HotpotQA. Stopping helps explain this difference. For every model, RAG+Laya most often failed by stopping with gold evidence missing. Under LLM control, most errors followed complete evidence or the hop limit (Section 4.4). The 0.8B gain also coincided with a different tool protocol: its `invalid_tool` rate was 0.290–0.550 with LLM control and two tools, but 0.000 with Laya and one tool. This does not explain the 2B gain, as its rate was at most 0.015. Held-out controller

quality (sufficient accuracy 0.9271, ECE 0.0626) was measured on simulated states. The 2B gain remains uncertain on 2Wiki and MuSiQue. Loop control should be evaluated together with the evidence source and tool protocol.

RQ2 (Cost). With retrieval, external control reduced tokens and hops in every configuration and lowered p50 latency for 0.8B–9B. Mean prompt tokens fell from 2,776–6,067 to 1,565–2,329 (Table 5). Laya calls averaged 16.0–20.8 ms for these models across conditions. Apart from the 0.8B model’s invalid calls, tool calls were reliable, with one error and no oom. Earlier stopping explains the savings. RAG+Laya most often stopped at hop 2 (Fig. 5), without an extra LLM call for the stopping decision. The LLM still generated the answer. Under its own control with retrieval, 2B almost always reached $H_{\max}$. At 27B, memory pressure complicates latency comparisons (run accounting is in Appendix A). These consistent savings justify external control only where accuracy is maintained (RQ3).

RQ3 (Scale). External control had opposite accuracy effects across model sizes, but saved tokens and hops throughout. Interaction point estimates were negative in all nine combinations with the three larger models and positive in all six with the two smaller models. CIs excluded zero in six of nine negative and two of six positive cases. Under its own control with retrieval, 2B almost never chose to stop. The 4B–27B models used more hops than under Laya (for example, 3.19–3.42 versus 2.53–2.55 on 2Wiki) and achieved higher EM. These runs do not identify a mechanism. For 4B–27B, Laya stopped with gold evidence missing on 194–230 questions that were then answered incorrectly. Possible causes include the fixed threshold of 0.5 with temperatures at the search limit (5.0), differences between simulated and retrieved states, and bridging facts lost through BUILDSTATE’s truncation. Smaller models’ gains are consistent with one available tool and difficulty judging when to stop. External control may help small models manage the loop while ending stronger models’ searches too early. This reversal was observed with one model family, loop, and threshold. Benefits should be evaluated at each size rather than assumed to carry over.

5.2. Limitations

With thinking disabled, we cannot compare Recall+LLM with a thinking-enabled baseline. Each corpus contains paragraphs from only 200 test questions, rather than full Wikipedia, potentially making retrieval easier than in open-domain settings. Each configuration ran once with one seed. Bootstrap intervals capture variation across questions, not runs. Differences up to 2 pp (four questions) should not be over-interpreted. We used only Qwen3.5 models and English datasets, excluding translated datasets, cloud models, human evaluation, a zero-shot Laya condition, and Laya as a passage re-ranker. Supporting-paragraph

annotations supplied weak labels to train one shared Laya on 800 questions per dataset. All results depend on the shared loop. Gold paragraphs serve as a proxy for sufficient evidence, although answers may not require all of them. All fitted temperatures reached the search limit (5.0), so the reported ECE does not establish optimal calibration.

6. Conclusion

We studied when calibrated external control should replace the generator’s control of multi-hop retrieval. The controller trained on weak supporting-paragraph labels met all pre-registered thresholds, with sufficient accuracy of 0.9271 and ECE of 0.0626. Retrieval contributed most, improving EM by up to 43.5 pp. **RQ1.** External control changed EM mainly under retrieval. Errors most often involved stopping before all gold evidence was retrieved. **RQ2.** With retrieval, Laya reduced prompt tokens, output tokens, and hops throughout. It eliminated invalid tool calls for 0.8B and lowered p50 latency for 0.8B–9B. **RQ3.** Under retrieval, Laya raised EM point estimates by 1.5–9.0 pp for 2B and 0.8B. The 4B–27B models achieved 3.5–15.5 pp higher EM under their own control. CIs support the 0.8B gain throughout and the 2B gain only on HotpotQA.

External control did not benefit every model. For the smallest models, it coincided with no invalid tool calls (0.8B), shorter retrieval loops, and higher EM point estimates at lower cost. For larger models, it stopped earlier, often before bridging evidence arrived, saving tokens and hops but lowering EM. This accuracy-efficiency trade-off depended on generator size and should be evaluated at each scale.

References

- [1] BAAI. BAAI/bge-reranker-v2-m3. <https://huggingface.co/BAAI/bge-reranker-v2-m3>, 2024. Hugging Face model card, Apache-2.0 license. Accessed 2026-09-29.
- [2] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-Embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2318–2335, Bangkok, Thailand, 2024. Association for Computational Linguistics.
- [3] Convai Innovations. Laya Multilingual. <https://huggingface.co/convaiinnovations/laya-multilingual>, 2026. Apache-2.0. Code: <https://github.com/NandhaKishorM/laya>; PyPI package laya v0.3.21. No accompanying paper. Accessed 2026-09-29.
- [4] Hongwen Ding and Yizheng Zhao. RAG-on-a-Diet:

- A reinforcement learning-based dynamic resource optimization framework for RAG. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33883–33904, San Diego, California, United States, 2026. Association for Computational Linguistics.
- [5] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *IEEE Transactions on Big Data*, 12(2):346–361, 2026.
 - [6] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, Barcelona, Spain (Online), 2020. International Committee on Computational Linguistics.
 - [7] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, Mexico City, Mexico, 2024. Association for Computational Linguistics.
 - [8] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992, Singapore, 2023. Association for Computational Linguistics.
 - [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 9459–9474. Curran Associates, Inc., 2020.
 - [10] Minghan Li, Junjie Zou, Xinxuan Lv, Chao Zhang, and Guodong Zhou. S2G-RAG: Structured sufficiency and gap judging for iterative retrieval-augmented QA. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25846–25862, San Diego, California, United States, 2026. Association for Computational Linguistics.
 - [11] Qwen Team. Qwen3.5: Towards native multimodal agents. <https://qwen.ai/blog?id=qwen3.5>, 2026. Official Qwen blog; no formal technical report for Qwen3.5 as of 2026-09-29. Ollama: <https://ollama.com/library/qwen3.5>. Accessed 2026-09-29.
 - [12] Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. DRAGIN: Dynamic retrieval augmented generation based on the real-time information needs of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12991–13013, Bangkok, Thailand, 2024. Association for Computational Linguistics.
 - [13] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
 - [14] Chi Wang and Qingyun Wu. AG2: Open-source AgentOS for AI agents (formerly AutoGen). <https://github.com/ag2ai/ag2>, 2024. Apache-2.0. Project site: <https://ag2.ai>. Accessed 2026-09-29.
 - [15] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *Proceedings of the First Conference on Language Modeling (COLM)*, 2024.
 - [16] Te-Lun Yang, Jyi-Shane Liu, Yuen-Hsien Tseng, Jyh-Shing Jang, Ming-Ching Chang, and Wei-Chao Chen. Enhancing LLM question answering with RAG through dense vector search and re-ranking. In *2025 IEEE International Conference on Advanced Visual and Signal-Based Systems (AVSS)*, pages 1–6, Tainan, Taiwan, 2025. IEEE.
 - [17] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, 2018. Association for Computational Linguistics.
 - [18] Zijun Yao, Weijian Qi, Liangming Pan, Shulin Cao, Linmei Hu, Weichuan Liu, Lei Hou, and Juanzi Li. SeaKR: Self-aware knowledge retrieval for adaptive retrieval augmented generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27022–27043, Vienna, Austria, 2025. Association for Computational Linguistics.

A. Implementation and Reproducibility Details

Software and hardware. Ollama 0.34.4 serves the language models (tags `qwen3.5:0.8b`, `:2b`, `:4b`, `:9b`, `:27b`) on an NVIDIA GeForce RTX 4090 under Ubuntu on WSL2 with Python 3.12. The embedder is BAAI/bge-m3, and the re-ranker is BAAI/bge-reranker-v2-m3. A FAISS flat inner-product index runs on the CPU. Because the 27B model ran fully on the GPU, BGE-M3, the re-ranker, and Laya also remained on the GPU in all 60 configurations. We obtain the datasets from Hugging Face (MuSiQue as `musique-ans`) and convert them to a common schema. AG2 1.1.0 could not disable thinking in our setting, so we implement the loop directly with the Ollama Python client.

Decision-layer training. Each simulated state presents a passage both as a full paragraph and as its supporting sentences (MuSiQue keeps the full paragraph). This exposes the layer to retrieved and recall-like evidence. We fine-tune the decision layer from `convaiinnovations/laya-multilingual` (322M parameters, input length 1024 tokens) on 32,451 simulated decisions and calibrate it on 3,996, in 10.8 min. Laya’s reinforcement learning [3] adds Gaussian noise to option logits for exploration. A strictly proper scoring rule rewards each sample against its weak label. Advantages use a group baseline, and a soft cross-entropy term guides training. We then fit one temperature per question type on the calibration split. All three reached 5.0, the upper end of the search range (Section 5.2).

Records and post-processing. For each question, we log the prediction, correctness, per-step trace (sub-questions, retrieved passage ids, Laya outputs), latency, tokens, and peak GPU memory. All results come from these records, the probe output, and the Laya evaluation report. Post-processing scripts reproduce the paired bootstrap intervals (seed 20261001), error taxonomy, and all result tables with identical bytes on every run. Questions that run out of GPU memory are retried once with a 4096-token context and recorded as `oom` if they fail again. Licenses are CC BY-SA 4.0 for HotpotQA, CC BY 4.0 for MuSiQue, MIT for BGE-M3, and Apache-2.0 for 2WikiMultihopQA, Laya, Qwen3.5, and the BGE re-ranker.

Run accounting. The single error (9B, 2Wiki, RAG+LLM) was a malformed tool-call XML reply. A question answered after k sub-questions has $k + 1$ hops unless it reaches $H_{\max}$. Laya never stopped at the first hop, when evidence was empty. Failed questions count hops only up to failure, lowering the 0.8B model’s mean under LLM control. Only two questions reached the 300s budget (2Wiki, 27B, RAG+LLM). They timed out three times in total, one of them twice. Both eventually finished after extra retries that no other question received. Thus, the final

Table 6. Wrong answers under retrieval by stage, per dataset (200 questions per row). Columns follow Table 4.

Dataset	Model	Condition	Inv.	T/O	Err.	Miss	Cov.	Stop	Oth.
HotpotQA	27B	RAG+LLM	0	0	0	1	50	14	0
		RAG+Laya	0	0	0	0	50	28	0
	9B	RAG+LLM	1	0	0	4	45	22	0
		RAG+Laya	0	0	0	2	48	29	0
	4B	RAG+LLM	0	0	0	8	58	5	0
		RAG+Laya	0	0	0	2	50	35	0
	2B	RAG+LLM	1	0	0	62	57	0	0
		RAG+Laya	0	0	0	15	61	29	0
	0.8B	RAG+LLM	58	0	0	62	34	0	0
		RAG+Laya	0	0	0	36	72	31	0
2Wiki	27B	RAG+LLM	0	0	0	9	51	3	0
		RAG+Laya	0	0	0	0	20	68	0
	9B	RAG+LLM	1	0	1	11	51	14	0
		RAG+Laya	0	0	0	0	30	77	0
	4B	RAG+LLM	0	0	0	14	65	1	0
		RAG+Laya	0	0	0	0	28	83	0
	2B	RAG+LLM	0	0	0	76	54	0	0
		RAG+Laya	0	0	0	2	35	90	0
	0.8B	RAG+LLM	78	0	0	78	22	1	0
		RAG+Laya	0	0	0	10	65	86	0
MuSiQue	27B	RAG+LLM	1	0	0	62	46	17	0
		RAG+Laya	0	0	0	9	30	98	0
	9B	RAG+LLM	1	0	0	74	41	20	0
		RAG+Laya	0	0	0	15	29	105	0
	4B	RAG+LLM	0	0	0	83	43	7	0
		RAG+Laya	0	0	0	14	30	112	0
	2B	RAG+LLM	0	0	0	177	4	0	0
		RAG+Laya	0	0	0	61	16	96	0
	0.8B	RAG+LLM	75	0	0	107	11	0	0
		RAG+Laya	0	0	0	62	17	101	0

`timeout_rate` is zero in every configuration.

B. Supplementary Results

Error taxonomy rules. Each wrong answer under retrieval receives one category, checked in the following order. A failed call has final status `invalid_tool`, `timeout`, or `error`. Evidence covered means that every gold supporting paragraph was retrieved at some hop. A premature stop leaves a gold paragraph missing when the last step stops the loop (Laya’s $p_{\text{suf}} \geq 0.5$ or the LLM’s `final_answer`). A retrieval miss leaves a gold paragraph missing even though the last step still retrieved. Remaining cases are classified as other. Table 6 reports counts per dataset, and Table 4 pools them.

A representative premature stop (HotpotQA, 27B). The question was "Harry Emerson Fosdick was called to serve as pastor at a church that opened its doors on what date?" The first hop retrieved the paragraph on Fosdick but not the one on Riverside Church. Laya then judged the evidence sufficient ($p_{\text{suf}} = 0.983$), and the model answered that the information was unavailable. Under its own control, the same model next asked "When did Riverside Church open its doors?" and answered correctly (October 5, 1930).

Intervals including zero; recall and retrieval effects. Under RAG, the CI for the Laya effect (Table 3, Panel B) includes zero for 2B on 2Wiki ($[-3.5, 6.5]$) and MuSiQue ($[0.0, 8.0]$), 9B on HotpotQA, and 27B on MuSiQue. Table 7 lists the controller effect under recall ($\Delta_{\text{Rec}} = \text{Re}$

Table 7. Recall effect Δ_{Rec} and retrieval effect Δ_{Ret} on EM (pp), estimate [paired bootstrap 95% CI], per model and dataset.

Model	Dataset	Δ_{Rec} [95% CI]	Δ_{Ret} [95% CI]
27B	HotpotQA	-2.0 [-5.5, 1.5]	37.5 [30.5, 45.0]
	2Wiki	1.5 [-2.0, 5.0]	36.5 [30.0, 43.5]
	MuSiQue	0.0 [-2.5, 2.5]	27.0 [20.0, 34.0]
9B	HotpotQA	0.0 [-3.5, 3.5]	43.0 [35.5, 50.5]
	2Wiki	0.0 [-3.5, 3.5]	35.0 [27.5, 42.5]
	MuSiQue	0.0 [-2.0, 2.0]	28.0 [21.5, 34.5]
4B	HotpotQA	0.5 [-3.0, 4.0]	43.5 [36.5, 50.5]
	2Wiki	2.5 [-2.5, 7.5]	35.5 [28.0, 43.5]
	MuSiQue	-1.0 [-2.5, 0.0]	30.0 [23.5, 37.0]
2B	HotpotQA	0.0 [-3.0, 3.0]	23.5 [16.5, 30.5]
	2Wiki	-2.0 [-6.5, 2.5]	10.5 [4.0, 17.0]
	MuSiQue	0.0 [-1.5, 1.5]	8.5 [4.5, 13.0]
0.8B	HotpotQA	5.5 [1.5, 9.5]	19.0 [13.0, 25.0]
	2Wiki	8.0 [3.5, 12.5]	3.5 [-1.0, 8.0]
	MuSiQue	0.5 [0.0, 1.5]	3.5 [1.0, 6.0]

Table 8. End-to-end p95 latency (ms) per model, condition, and dataset.

Model	Condition	HotpotQA	2Wiki	MuSiQue
27B	Recall+LLM	17286	13314	33059
	RAG+LLM	25500	18983	24279
	Recall+Laya	32216	24109	39684
	RAG+Laya	20212	16338	22110
9B	Recall+LLM	5712	4324	6232
	RAG+LLM	3530	5064	3729
	Recall+Laya	5771	5025	5397
	RAG+Laya	2982	2502	3463
4B	Recall+LLM	5324	4365	5657
	RAG+LLM	3296	4175	3172
	Recall+Laya	5145	4570	4969
	RAG+Laya	2587	2205	2749
2B	Recall+LLM	3452	3433	3701
	RAG+LLM	2818	3782	2146
	Recall+Laya	2778	3149	3311
	RAG+Laya	1966	1643	2063
0.8B	Recall+LLM	2237	2175	2247
	RAG+LLM	2723	3302	2053
	Recall+Laya	2092	1847	1999
	RAG+Laya	1935	2012	1768

call+Laya minus Recall+LLM) and the retrieval effect (Δ_{Ret} = RAG+LLM minus Recall+LLM) with paired bootstrap 95% CIs. The Δ_{Rec} CI excludes zero only for 0.8B on HotpotQA and 2Wiki. The Δ_{Ret} CI excludes zero in every pair except 0.8B on 2Wiki ($[-1.0, 8.0]$).

Costs under recall and tail latency. Recall+Laya also uses fewer prompt tokens than Recall+LLM (Table 5). Its stopping decision requires no extra LLM call, which partly explains this reduction. Under recall, p50 latency falls on every dataset only for 0.8B and 2B (for example, 1277 versus 2733 ms for 2B on HotpotQA). Table 8 reports end-to-end p95 latency alongside the p50 results in Table 5.